Deep Research: Avatrada Ui 9 Topic 007

Engineering Report: Trading Platform Backend Architecture

To: Lead Architect **From**: Senior Backend Architect **Date**: October 26, 2023 **Subject**: Deep-Dive on Core Backend Components for GCP-Based Trading Platform

This report provides a detailed architectural breakdown and implementation strategy for the core backend components of the new trading platform, as per Master Research Prompt 7. The analysis covers the WebSocket gateway, the asynchronous task processing system, security architecture for hardware 2FA, and the immutable audit trail database schema.

---

1. FastAPI WebSocket Gateway for Real-Time Data

1.1. Technical Deconstruction

The objective is to create a stateful WebSocket gateway capable of managing thousands of concurrent connections. The system must support channel-based subscriptions, where clients subscribe to specific data streams (e.g., stock tickers like 'AAPL'). A central `ConnectionManager` class will maintain the state of active connections and their subscriptions, enabling targeted message broadcasting. This architecture is fundamental for pushing real-time market data, order status updates, and notifications to the frontend with low latency.

1.2. Implementation Strategy

We will implement a `ConnectionManager` class within our FastAPI application. This class will use a dictionary to map channel names (tickers) to a list of active `WebSocket` connection objects.

Key Libraries: * `fastapi`: The core web framework. * `websockets`: The underlying library used by FastAPI for WebSocket handling. * `asyncio`: For concurrent I/O operations.

Boilerplate Code:

```
# main.py
import asyncio
from collections import defaultdict
from typing import Dict, List

from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

class ConnectionManager:
    """
    Manages active WebSocket connections and channel subscriptions.
    - active_connections: Maps a channel (e.g., ticker) to a list of WebSocket
clients.
    """
    def __init__(self):
        self.active_connections: Dict[str, List[WebSocket]] = defaultdict(list)

    async def connect(self, websocket: WebSocket, channel: str):
        """Accept a new connection and subscribe it to a channel."""
        await websocket.accept()
        self.active_connections[channel].append(websocket)
        print(f"Client connected. Subscribed to channel: {channel}")

    def disconnect(self, websocket: WebSocket, channel: str):
        """Remove a disconnected client from its channel."""
        self.active_connections[channel].remove(websocket)
        print(f"Client disconnected from channel: {channel}")

    async def broadcast_to_channel(self, message: str, channel: str):
        """Broadcast a message to all clients subscribed to a specific
channel."""
        if channel in self.active_connections:
            # Create a list of tasks to send messages concurrently
```

```
            tasks = [
                connection.send_text(message)
                for connection in self.active_connections[channel]
            ]
            await asyncio.gather(*tasks, return_exceptions=False) # Set to True
to handle individual send errors

manager = ConnectionManager()

@app.websocket("/ws/market-data/{ticker}")
async def websocket_endpoint(websocket: WebSocket, ticker: str):
    """
    WebSocket endpoint for clients to subscribe to market data for a specific
ticker.
    """
    await manager.connect(websocket, ticker)
    try:
        while True:
            # We can listen for messages from the client if needed
            # For a pure broadcast system, this loop just keeps the connection
alive.
            data = await websocket.receive_text()
            # Example of echoing back, not typically used for market data
            await websocket.send_text(f"Message text was: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket, ticker)

# Example of an external process (e.g., a data feed handler) pushing data
async def price_feed_simulator():
    """Simulates a market data feed pushing updates."""
    import random
    tickers = ['AAPL', 'TSLA', 'GOOG']
    while True:
        await asyncio.sleep(1)
        ticker = random.choice(tickers)
        price = f'{{"ticker": "{ticker}", "price": {random.uniform(100, 500):.
2f}}}'
        print(f"Broadcasting update: {price}")
        await manager.broadcast_to_channel(price, ticker)
```

```
@app.on_event("startup")
async def startup_event():
    """Create the background task for the price feed on startup."""
    asyncio.create_task(price_feed_simulator())
```

1.3. Critical Analysis

- **Failure Mode (Single Point of Failure)**: The provided `ConnectionManager` stores connection state in the memory of a single Python process. If this process crashes or the server needs to be restarted, all WebSocket connections are dropped. Furthermore, this design does not scale horizontally. If we run multiple instances of the FastAPI application behind a load balancer, a message published to instance A will not reach clients connected to instance B.
- **Optimization (Scaling with Redis Pub/Sub)**: To overcome this, we must externalize the messaging layer. Each FastAPI instance becomes stateless regarding messaging.
    1. **Publish**: When the data feed needs to send an update for 'AAPL', it publishes the message to a Redis 'AAPL' channel.
    2. **Subscribe**: Each FastAPI instance maintains a long-running task that subscribes to all relevant Redis channels.
    3. **Broadcast**: When an instance receives a message from Redis for the 'AAPL' channel, it then uses its local `ConnectionManager` to broadcast that message to the clients *it* is responsible for. This allows for near-infinite horizontal scaling of the WebSocket gateway.
- **Edge Case (Connection Handling)**: The `broadcast_to_channel` function should handle potential `ConnectionClosed` errors if a client disconnects between the check and the send. The `asyncio.gather` with `return_exceptions=True` can help manage this gracefully without crashing the broadcast loop.
- **Non-Blocking I/O**: The source document mentions non-blocking database writes. This is critical. While broadcasting prices, an audit log write must not block the event loop. Using an async database driver like `asyncpg` for PostgreSQL is essential. `python # Conceptual example inside a data processing function async def process_and_log_data(data, channel): #`

```
Non-blocking write to the audit log db_write_task = asyncio.create_task(db.log_event(...)) # Concurrently broadcast to clients broadcast_task = asyncio.create_task(manager.broadcast_to_channel(data, channel)) # Await both tasks to complete await asyncio.gather(db_write_task, broadcast_task)
```

---

2. Celery + Redis for Asynchronous Backtesting

2.1. Technical Deconstruction

The goal is to offload computationally intensive backtesting jobs from the main API process to prevent blocking user requests. The architecture uses Celery as the distributed task queue and Redis as both the message broker (to send tasks to workers) and the result backend (to store task status and results). This allows the API to immediately respond to the user with a `task_id` while a dedicated worker handles the computation. The frontend can then use this `task_id` to query for progress updates, which are pushed in real-time via the WebSocket gateway.

2.2. Implementation Strategy

Architecture Flow: 1. **Request**: User submits a backtest request via a FastAPI POST endpoint. 2. **Dispatch**: FastAPI validates the request and dispatches a task to a Celery queue (e.g., `heavy_compute_queue`) using `run_backtest.delay(...)`. It immediately returns the `task_id` to the user. 3. **Execution**: A dedicated Celery worker, subscribed to `heavy_compute_queue`, picks up the task and begins the backtest. 4. **Progress Reporting**: During execution, the worker updates its state using `self.update_state()`. This state, including a custom progress percentage, is stored in the Redis result backend. 5. **Real-time Update**: **(Advanced Pattern)** Upon updating its state, the worker also publishes a message to a Redis Pub/Sub channel (e.g., `task_progress:{task_id}`). 6. **Frontend Notification**: The FastAPI WebSocket gateway is subscribed to these Redis channels and pushes the progress update to the specific client who initiated the task. 7. **Result Storage**: Upon completion, the final result (e.g., a

JSON report or a pointer to a file in Google Cloud Storage) is stored in the Redis result backend.

Boilerplate Code:

```
 # celery_worker/tasks.py
import time
import redis
from celery import Celery
from celery.utils.log import get_task_logger

logger = get_task_logger(__name__)

# Configure Celery
# The broker is where tasks are sent, the backend is where results are stored.
celery_app = Celery(
    'tasks',
    broker='redis://localhost:6379/0',
    backend='redis://localhost:6379/0'
)

# Redis client for custom Pub/Sub notifications
redis_client = redis.Redis(host='localhost', port=6379, db=0)

@celery_app.task(bind=True)
def run_backtest(self, user_id: str, strategy_params: dict):
    """
    A long-running task that simulates a historical backtest.
    Updates its progress periodically.
    """
    total_steps = 100
    task_id = self.request.id
    logger.info(f"Starting backtest {task_id} for user {user_id}")

    for i in range(total_steps):
        # --- HEAVY COMPUTATION LOGIC HERE ---
        time.sleep(0.5) # Simulate work
        # --- END HEAVY COMPUTATION ---

        progress = {
```

```
            'current': i + 1,
            'total': total_steps,
            'status': 'Running backtest...'
        }

        # Update Celery's official state
        self.update_state(state='PROGRESS', meta=progress)

        # (Advanced) Publish real-time update to Redis Pub/Sub for WebSockets
        # The channel is specific to this task, so only the relevant user gets
it.
        channel = f"task_progress:{task_id}"
        redis_client.publish(channel, str(progress))

    result_data = {'pnl': 12345.67, 'sharpe_ratio': 1.8}
    return result_data

# fastapi_app/main.py
from fastapi import FastAPI
from celery.result import AsyncResult
# from celery_worker.tasks import run_backtest # Import the task

app = FastAPI()

@app.post("/backtest")
async def start_backtest_task(params: dict):
    """Endpoint to start a backtest."""
    # Assume user_id is retrieved from auth token
    user_id = "user_123"
    task = run_backtest.delay(user_id, params)
    return {"task_id": task.id}

@app.get("/backtest/status/{task_id}")
async def get_backtest_status(task_id: str):
    """Endpoint for frontend to poll for status (fallback if WebSockets
fail)."""
    task_result = AsyncResult(task_id, app=celery_app)
    response = {
        "task_id": task_id,
        "status": task_result.state,
```

```
        "progress": task_result.info,
    }
    if task_result.successful():
        response['result'] = task_result.get()
    return response
```

2.3. Critical Analysis

- **Worker Separation**: The source document correctly identifies the need for worker separation. CPU-bound tasks like backtesting should be routed to a queue consumed by workers running on high-CPU GCP machine types (e.g., `c2-standard-8`). IO-bound tasks like fetching news from multiple APIs should go to a separate queue consumed by workers that can handle high concurrency (e.g., using `gevent` or `eventlet` execution pools). This prevents a long-running calculation from starving time-sensitive I/O tasks. This is configured in Celery using task routing.
- **Failure Mode (Result Size)**: Storing large backtest results (e.g., thousands of simulated trades) directly in Redis is an anti-pattern. Redis is an in-memory database and is not suited for large object storage.
- **Optimization (Result Storage in GCS)**: The final result of a backtest should be serialized (e.g., as a Parquet or JSON file) and uploaded to a Google Cloud Storage (GCS) bucket. The Celery task should then return the GCS URI of the result file. This is more scalable, cost-effective, and durable.
- **Edge Case (Task Idempotency)**: What happens if a task is triggered twice due to a network retry? The backtest might run twice, wasting resources. Tasks should be designed to be idempotent where possible. For example, the system could check if a result for the exact same set of parameters already exists before starting a new computation.

---

3. Hardware Key 2FA (WebAuthn/YubiKey) Integration

3.1. Technical Deconstruction

WebAuthn is a W3C standard for secure, public-key-based authentication. It replaces or augments passwords with hardware authenticators (like YubiKeys) or platform authenticators (like Windows Hello or Touch ID). The flow is a challenge-response protocol designed to be phishing-resistant, as the credential is bound to the origin (domain name).

The flow consists of two main phases: 1. **Registration Ceremony**: The user associates their hardware key with their account. The server generates a challenge, the key signs it, and the server stores the resulting public key credential. 2. **Authentication Ceremony**: To log in, the user proves possession of the key. The server sends a new challenge, the key signs it, and the server verifies the signature using the previously stored public key.

3.2. Implementation Strategy

We will use a dedicated Python library to handle the complex FIDO2/WebAuthn server-side logic.

Key Libraries: * `webauthn`: A popular and robust library for implementing the Relying Party (server) side of WebAuthn. * `fastapi`: For the API endpoints. * `passlib`: For password hashing (as WebAuthn is often a second factor).

Conceptual Implementation Flow:

```
# main.py - Conceptual Endpoints
from fastapi import FastAPI, Request, Depends
from webauthn import generate_registration_options, verify_registration_response
from webauthn import generate_authentication_options,
verify_authentication_response
# Assume user management and credential storage functions exist (e.g.,
db.get_user, db.save_credential)
```

```
app = FastAPI()

# --- 1. REGISTRATION ---

@app.post("/webauthn/register/start")
async def start_registration(user: User = Depends(get_current_user)):
    """Generate and return registration options (challenge) for the frontend."""
    options = generate_registration_options(
        rp_id="trading.avatrada.com", # Your domain
        rp_name="Avatrada Trading",
        user_id=str(user.id),
        user_name=user.username,
    )
    # Store the challenge in the user's session or a short-lived cache
    request.session['webauthn_challenge'] = options['challenge']
    return options

@app.post("/webauthn/register/verify")
async def verify_registration(request: Request, user: User =
Depends(get_current_user)):
    """Verify the signed challenge from the authenticator."""
    body = await request.json()
    challenge = request.session.pop('webauthn_challenge')

    verification = verify_registration_response(
        credential=body,
        expected_challenge=challenge,
        expected_origin="https://trading.avatrada.com",
        expected_rp_id="trading.avatrada.com",
    )
    # On success, save verification.credential_public_key and
verification.sign_count to the database, associated with the user.
    db.save_credential_for_user(user.id, verification)
    return {"verified": True}

# --- 2. AUTHENTICATION ---

@app.post("/webauthn/login/start")
async def start_login(username: str):
    """Generate an authentication challenge for a given user."""
```

```
    user_credentials = db.get_credentials_for_user(username)
    if not user_credentials:
        raise HTTPException(status_code=404, detail="User not found or no
credentials registered")

    options = generate_authentication_options(
        rp_id="trading.avatrada.com",
        allow_credentials=[{"type": "public-key", "id": cred.id} for cred in
user_credentials],
    )
    request.session['webauthn_challenge'] = options['challenge']
    return options

@app.post("/webauthn/login/verify")
async def verify_login(request: Request, username: str):
    """Verify the signed authentication challenge."""
    body = await request.json()
    challenge = request.session.pop('webauthn_challenge')
    user_credential = db.get_credential_by_id(body['id'])

    verification = verify_authentication_response(
        credential=body,
        expected_challenge=challenge,
        expected_origin="https://trading.avatrada.com",
        expected_rp_id="trading.avatrada.com",
        credential_public_key=user_credential.public_key,
        credential_current_sign_count=user_credential.sign_count,
    )
    # IMPORTANT: Update the sign_count in the database to the new value.
    db.update_sign_count(user_credential.id, verification.new_sign_count)

    # If verification is successful, issue a JWT or session token.
    return {"verified": True, "token": create_access_token(username)}
```

3.3. Critical Analysis

- **Critical Component (Sign Count)**: The `sign_count` is a counter maintained by the authenticator and verified by the server. It is a mandatory security measure to prevent replay attacks using a cloned

authenticator. The server **must** store the last-seen `sign_count` and ensure that the count in any new authentication assertion is strictly greater than the stored value.

- **Secret Management**: The source document mentions Google Secret Manager. This is the correct approach for storing sensitive configuration, such as the database connection string, API keys for data providers, and any secret keys used for signing session cookies or JWTs. These secrets should be fetched programmatically at application startup and injected as environment variables, never hardcoded.
- **Failure Mode (Key Loss)**: A robust implementation must include a recovery mechanism for users who lose their hardware key. This could involve pre-generated one-time recovery codes, or a support-driven identity verification process. This fallback mechanism itself must be highly secure.
- **User Experience**: The frontend implementation is non-trivial. It requires using the browser's `navigator.credentials` API (`create()` for registration, `get()` for authentication). Clear instructions and error handling must be provided to the user.

---

4. PostgreSQL Schema for Immutable Audit Trail

4.1. Technical Deconstruction

An immutable audit trail is a log of all significant events within the system that, once written, cannot be altered or deleted. This is a non-negotiable requirement for financial systems, serving compliance, security forensics, and debugging purposes. We will use PostgreSQL to implement this, leveraging its `JSONB` data type for flexible event payloads and `TIMESTAMPTZ(6)` for microsecond-precision timestamps. Immutability will be enforced at both the application and database levels.

4.2. Implementation Strategy

We will create a single, partitioned table named `audit_log`. A single table simplifies chronological queries across all event types. Partitioning by time is essential for managing performance as the table grows to billions of rows.

SQL Schema Definition:

```
 -- Create a custom ENUM type for known event types for data integrity.
CREATE TYPE audit_event_type AS ENUM (
    'USER_LOGIN_SUCCESS',
    'USER_LOGIN_FAIL',
    'ORDER_CREATED',
    'ORDER_CANCELLED',
    'ORDER_EXECUTED',
    'SYSTEM_ALERT_HIGH_CPU',
    'BACKTEST_STARTED',
    'BACKTEST_COMPLETED'
);

-- Create the main audit log table, partitioned by month.
CREATE TABLE audit_log (
    id BIGSERIAL,
    event_timestamp TIMESTAMPTZ(6) NOT NULL DEFAULT (now() AT TIME ZONE 'utc'),
    event_type audit_event_type NOT NULL,
    actor_id TEXT, -- Can be user_id, system_process_name, etc.
    entity_id TEXT, -- Can be order_id, asset_id, etc.
    client_ip INET, -- The IP address of the request originator
    payload JSONB, -- Flexible field for event-specific data
    PRIMARY KEY (id, event_timestamp)
) PARTITION BY RANGE (event_timestamp);

-- Create partitions for the next few months as an example.
-- This should be automated via a cron job or maintenance script.
CREATE TABLE audit_log_y2023m11 PARTITION OF audit_log
    FOR VALUES FROM ('2023-11-01 00:00:00+00') TO ('2023-12-01 00:00:00+00');

CREATE TABLE audit_log_y2023m12 PARTITION OF audit_log
    FOR VALUES FROM ('2023-12-01 00:00:00+00') TO ('2024-01-01 00:00:00+00');

-- Create indexes for common query patterns.
CREATE INDEX idx_audit_log_actor_id ON audit_log (actor_id);
CREATE INDEX idx_audit_log_entity_id ON audit_log (entity_id);
CREATE INDEX idx_audit_log_event_type ON audit_log (event_type);

-- Enforce Immutability at the Database Level
```

```
-- 1. Create a function that raises an exception.
CREATE OR REPLACE FUNCTION prevent_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Modifications to the audit_log are forbidden.';
END;
$$ LANGUAGE plpgsql;

-- 2. Create a trigger that calls this function on any UPDATE or DELETE attempt.
CREATE TRIGGER trg_audit_log_immutable
BEFORE UPDATE OR DELETE ON audit_log
FOR EACH ROW EXECUTE FUNCTION prevent_modification();
```

Example `payload` JSONB content: * For `ORDER_CREATED`: `{"symbol": "AAPL", "quantity": 100, "order_type": "LIMIT", "limit_price": 175.50}` * For `SYSTEM_ALERT_HIGH_CPU`: `{"hostname": "worker-prod-c2-1", "metric": "cpu_utilization", "value": 0.95}`

4.3. Critical Analysis

- **Partition Management**: The biggest operational challenge with this design is partition management. A scheduled process (e.g., a Cloud Function triggered by Cloud Scheduler) must run periodically to create future partitions and potentially detach/archive old ones to cold storage (like GCS) to manage costs and keep the "hot" queryable dataset at a reasonable size.
- **Performance**: While `JSONB` is flexible, querying nested fields can be slower than querying indexed, structured columns. If certain fields within the payload are queried very frequently (e.g., `payload->>'symbol'`), it is worth creating a GIN index on the `payload` column (`CREATE INDEX idx_audit_log_payload_gin ON audit_log USING GIN (payload);`) or even promoting that field to its own top-level, indexed column.
- **Defense in Depth**: In addition to the database trigger, the application's database user role should be granted only `INSERT` and `SELECT` permissions on the `audit_log` table. This provides an additional layer of security, preventing accidental or malicious modification attempts from the application code itself.

- **Timestamp Precision**: Using `TIMESTAMPTZ(6)` is critical for establishing an unambiguous sequence of events, especially in a distributed system where events can occur in very quick succession across different services. Storing everything in UTC is a non-negotiable best practice.